KU LEUVEN

FACULTEIT INDUSTRIËLE
INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Labo 5

Labo Geavanceerde Computerarchitectuur

Kjell NAESSENS
Beau FORREZ

Labo verantwoordelijke: T. Feys

Academiejaar 2025-2026

1 Context

In dit labo onderzoeken we het verschil tussen synchrone en asynchrone operatie bij het gebruik van een GPU. We genereren op de CPU telkens een array en voeren onmiddellijk daarna een bewerking uit op deze array op de GPU. Bij de synchrone operatie wachten we met het genereren van de volgende array tot de GPU klaar is met de bewerking. Bij de asynchrone operatie wordt de bewerking op de GPU uitgevoerd terwijl de CPU al onmiddellijk begint met de volgende array te genereren. Hierdoor moet de CPU niet wachten op de GPU.

2 Methodologie

2.1 Synchrone uitvoering

Sequentieel wordt een array gegenereerd op de CPU en wordt een bewerking uitgevoerd op deze array. Deze sequentie wordt 4 keer uitgevoerd, telkens met een andere bewerking in de GPU kernel.

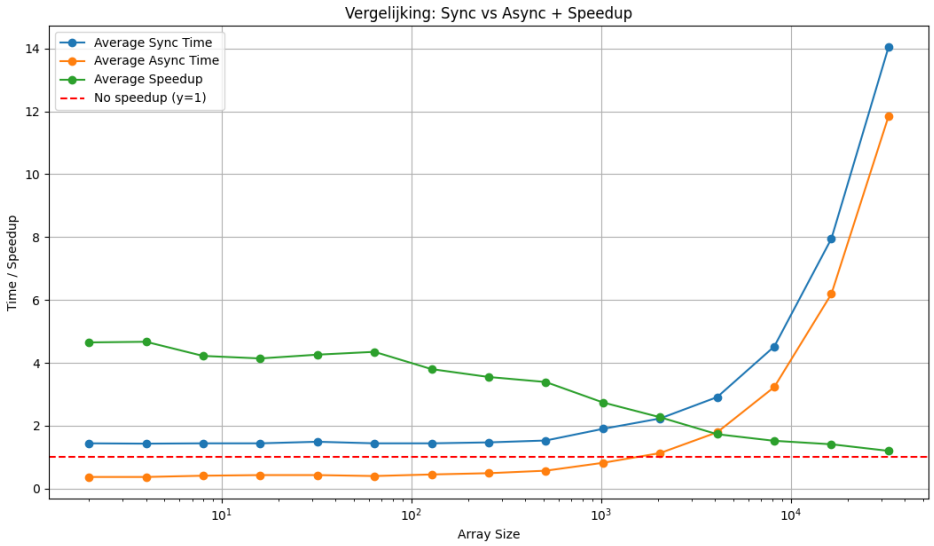
2.2 Asynchrone uitvoering

Er wordt opnieuw sequentieel een array gegenereerd op de CPU, maar nu wordt telkens na het genereren van deze arrays een kernel gestart die asynchroon een bewerking zal uitvoeren op de array. De CPU wacht niet meer op de GPU om de volgende array te genereren.

3 Observaties en resultaten

We voeren de volledige operatie 1000 keer uit om een gemiddelde tijd van beide methoden te bekomen. Hieruit berekenen we ook de speedup door gebruik te maken van dit gemiddelde. De eerste run werd genegeerd om een cold start te voorkomen.

We merkten een afhankelijkheid van de array grootte. Dit komt doordat de taak eerder CPU-bound is bij grote array sizes. Figuur 1 toont de bekomen resultaten.



Figuur 1: Speed up in functie van array size

4 Besluit

Wanneer we gebruik maken van asynchrone uitvoering zien we een speedup tot een factor 4. Naargelang de input array grootte toeneemt, daalt de speed up wel. Dit komt omdat de taken meer CPU-bound worden: de uitvoeringstijd voor het genereren van de arrays worden vele malen groter dan de uitvoeringstijden van de bewerkingen op de arrays. Hierdoor worden deze bewerkingen vrijwel verwaarloosbaar en zien we geen speedup meer.

Voor taken die niet CPU-bound zijn, zien we het voordeel om asynchroon te werken.

5 Appendix

```cpp
#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <chrono>
#include <algorithm>
#include <cstdlib>
#include <iomanip>
#include <numeric>

// Array size and block size
#define N (1 << 8)
#define THREADS 256
#define NUM_RUNS 1000 // loops

// Operation selection
__device__ int op(int a, int b, int operation) {
    if (operation == 0) return a + b;
    else if (operation == 1) return a * b;
    else if (operation == 2) return min(a, b);
    else return max(a, b);
}


// General-purpose reduction kernel.
__global__ void reduction_kernel(int* input, int* output, int numElements, int
    operation) {
    extern __shared__ int sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * (blockDim.x * 2) + threadIdx.x;

    if (i + blockDim.x < numElements) {
        sdata[tid] = op(input[i], input[i + blockDim.x], operation);
    } else if (i < numElements) {
        sdata[tid] = input[i];
    } else {
        if (operation == 0) sdata[tid] = 0;
        else if (operation == 1) sdata[tid] = 1;
        else if (operation == 2) sdata[tid] = 0x7FFFFFFF;
        else sdata[tid] = 0x80000000;
    }
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = op(sdata[tid], sdata[tid + s], operation);
        }
        __syncthreads();
    }
```

```
48
49      if (tid == 0) {
50          output[blockIdx.x] = sdata[0];
51      }
52  }
53
54  void generate_array(std::vector<int>& arr, int op_type) {
55      if (op_type == 1) {
56          std::generate(arr.begin(), arr.end(), []() { return (rand() % 3) + 1;
                });
57      } else if (op_type == 2) {
58          std::generate(arr.begin(), arr.end(), []() { return (rand() % 10000) +
                1; });
59      } else {
60          std::generate(arr.begin(), arr.end(), []() { return (rand() % 100) + 1;
                 });
61      }
62  }
63
64  int reduce_array_sync(int* d_input, int numElements, int operation) {
65      int blocks = (numElements + THREADS * 2 - 1) / (THREADS * 2);
66      int* d_intermediate;
67      cudaMalloc(&d_intermediate, blocks * sizeof(int));
68
69      int currentElements = numElements;
70      int* inputPtr = d_input;
71      int* outputPtr = d_intermediate;
72
73      while (blocks > 1) {
74          reduction_kernel<<<blocks, THREADS, THREADS * sizeof(int)>>>(inputPtr,
                outputPtr, currentElements, operation);
75          cudaDeviceSynchronize();
76          currentElements = blocks;
77          std::swap(inputPtr, outputPtr);
78          blocks = (currentElements + THREADS * 2 - 1) / (THREADS * 2);
79      }
80
81      reduction_kernel<<<1, THREADS, THREADS * sizeof(int)>>>(inputPtr,
            d_intermediate, currentElements, operation);
82      cudaDeviceSynchronize();
83
84      int h_result;
85      cudaMemcpy(&h_result, d_intermediate, sizeof(int), cudaMemcpyDeviceToHost);
86      cudaFree(d_intermediate);
87      return h_result;
88  }
89
90  void reduce_array_on_stream(int* d_input, int* d_intermediate, int*
        d_final_result, int numElements, int operation, cudaStream_t stream) {
91      int blocks = (numElements + THREADS * 2 - 1) / (THREADS * 2);
92      int currentElements = numElements;
```

```
    int* inputPtr = d_input;
    int* outputPtr = d_intermediate;

    while (blocks > 1) {
        reduction_kernel<<<blocks, THREADS, THREADS * sizeof(int), stream>>>(
            inputPtr, outputPtr, currentElements, operation);
        currentElements = blocks;
        std::swap(inputPtr, outputPtr);
        blocks = (currentElements + THREADS * 2 - 1) / (THREADS * 2);
    }
    reduction_kernel<<<1, THREADS, THREADS * sizeof(int), stream>>>(inputPtr,
        d_final_result, currentElements, operation);
}

// --- Synchronous Execution ---
void run_sync(std::vector<std::vector<int>>& arrays) {
    for (int op_type = 0; op_type < 4; ++op_type) {
        for (int i = 0; i < 4; ++i) {
            generate_array(arrays[i], op_type);
            int* d_arr;
            cudaMalloc(&d_arr, N * sizeof(int));
            cudaMemcpy(d_arr, arrays[i].data(), N * sizeof(int),
                cudaMemcpyHostToDevice);
            reduce_array_sync(d_arr, N, op_type);
            cudaFree(d_arr);
        }
    }
}

// --- Asynchronous Execution ---
void run_async(std::vector<std::vector<int>>& arrays) {
    cudaStream_t streams[4];
    for (int i = 0; i < 4; ++i) cudaStreamCreate(&streams[i]);

    int* d_arrs[4];
    int* d_intermediates[4];
    int* d_results[4];
    int h_results[4];
    int intermediate_size = ((N + THREADS * 2 - 1) / (THREADS * 2)) * sizeof(
        int);

    for (int i = 0; i < 4; ++i) {
        cudaMalloc(&d_arrs[i], N * sizeof(int));
        cudaMalloc(&d_intermediates[i], intermediate_size);
        cudaMalloc(&d_results[i], sizeof(int));
    }

    for (int op_type = 0; op_type < 4; ++op_type) {
        for (int i = 0; i < 4; ++i) {
            generate_array(arrays[i], op_type);
```

```
            cudaMemcpyAsync(d_arrs[i], arrays[i].data(), N * sizeof(int),
                cudaMemcpyHostToDevice, streams[i]);
            reduce_array_on_stream(d_arrs[i], d_intermediates[i], d_results[i],
                 N, op_type, streams[i]);
            cudaMemcpyAsync(&h_results[i], d_results[i], sizeof(int),
                cudaMemcpyDeviceToHost, streams[i]);
        }
        cudaDeviceSynchronize(); // Wacht op alle streams voor deze operatie
    }

    // Cleanup
    for (int i = 0; i < 4; ++i) {
        cudaFree(d_arrs[i]);
        cudaFree(d_intermediates[i]);
        cudaFree(d_results[i]);
        cudaStreamDestroy(streams[i]);
    }
}

int main() {
    std::srand(static_cast<unsigned>(std::time(nullptr)));

    std::vector<double> sync_times, async_times, speedups;

    // --- WARM-UP (niet gemeten) ---
    {
        std::cout << "Warming up GPU (first run, not counted)...\n";
        std::vector<std::vector<int>> warmup_arrays(4, std::vector<int>(N));
        run_sync(warmup_arrays);   // Sync warm-up
        run_async(warmup_arrays);  // Async warm-up (streams, kernels, etc.)
        cudaDeviceSynchronize();
        std::cout << "Warm-up complete.\n\n";
    }

    std::cout << "Running " << NUM_RUNS << " measured iterations...\n\n";

    // --- GEMETEN RUNS (NUM_RUNS keer) ---
    for (int run = 0; run < NUM_RUNS; ++run) {
        std::vector<std::vector<int>> arrays(4, std::vector<int>(N));

        // --- Sync ---
        auto start_sync = std::chrono::high_resolution_clock::now();
        run_sync(arrays);
        cudaDeviceSynchronize();
        auto end_sync = std::chrono::high_resolution_clock::now();
        double sync_ms = std::chrono::duration<double, std::milli>(end_sync -
            start_sync).count();

        // --- Async ---
        auto start_async = std::chrono::high_resolution_clock::now();
        run_async(arrays);
```

```
186         auto end_async = std::chrono::high_resolution_clock::now();
187         double async_ms = std::chrono::duration<double, std::milli>(end_async -
                start_async).count();
188
189         double speedup = sync_ms / async_ms;
190
191         sync_times.push_back(sync_ms);
192         async_times.push_back(async_ms);
193         speedups.push_back(speedup);
194
195         std::cout << "Run " << std::setw(2) << (run + 1)
196                   << " | Sync: " << std::fixed << std::setprecision(2) <<
                        sync_ms << " ms"
197                   << " | Async: " << async_ms << " ms"
198                   << " | Speedup: " << std::setprecision(2) << speedup << "x\n"
                        ;
199     }
200
201     // --- Gemiddelden (alleen over de NUM_RUNS gemeten runs) ---
202     double avg_sync = std::accumulate(sync_times.begin(), sync_times.end(),
            0.0) / NUM_RUNS;
203     double avg_async = std::accumulate(async_times.begin(), async_times.end(),
            0.0) / NUM_RUNS;
204     double avg_speedup = std::accumulate(speedups.begin(), speedups.end(), 0.0)
             / NUM_RUNS;
205
206     std::cout << "\n" << std::string(60, '=') << "\n";
207     std::cout << "AVERAGE OVER " << NUM_RUNS << " RUNS (warm-up excluded):\n";
208     std::cout << std::string(60, '-') << "\n";
209     std::cout << "Average Sync Time : " << std::fixed << std::setprecision(2)
            << avg_sync << " ms\n";
210     std::cout << "Average Async Time: " << avg_async << " ms\n";
211     std::cout << "Average Speedup   : " << avg_speedup << "x\n";
212     std::cout << std::string(60, '=') << "\n";
213
214     return 0;
215 }
```